DAY 21 SYSTEM DESIGN JOURNEY 🚀

EVENT SOURCING, SNAPSHOTTING, AND AGGREGATE REBUILD

Make events the source of truth. Derive state, replay history, and rebuild with confidence.

THE BIG IDEA

Store every business change as an immutable event. Current state is derived by replaying events in order. You get **auditability**, **replay**, and **temporal debugging** for free.

1. APPEND ONLY

Events are immutable and appended, never updated or deleted.

2. SOURCE OF TRUTH

Events are the source of truth. State is derived by replaying them.

3. REBUILDABLE

Any state can be reconstructed by replaying events in order.

4. AUDITABLE

Full history of changes enables audit, compliance and debugging.

5. SNAPSHOTS

Snapshots speed up rehydration by saving state at a known version.

6. CONSISTENCY

Optimistic concurrency with expected version prevents lost updates.

HOW IT WORKS

1. COMMAND

Client

Send Command

(deposit / withdraw /open account)

→

2. LOAD AGGREGATE

Load from Snapshot (if any)

+

Replay Tail Events (from last snapshot version)

→

3. VALIDATE & APPLY

Validate Business Rules

↓

Apply Command Produce New Event(s)

→

4. APPEND EVENTS

Append to Event Store

With Expected Version (Optimistic Concurrency)

→

5. NEW STATE (DERIVED)

State can be rebuilt anytime by replaying events

Deterministic Rebuild = Reliable Consistency

EVENT STORE & SNAPSHOT FLOW

Event Stream (per Aggregate)

v1	v2	v3	v4	v5	...
AccountOpened	MoneyDeposited	MoneyWithdrawn	MoneyDeposited	MoneyWithdrawn	

Old ⟵ Time / Version ⟶ New

SNAPSHOT STORE

Snapshot at version 3

Balance = 700
Version = 3

→ Rehydrate:
Load Snapshot (v3)
Replay events v4, v5...

→ Current State

Balance = 500
Version = 5

AGGREGATE REBUILD EXAMPLE (ACCOUNT)

```
Events:
v1  AccountOpened                  balance = 0
v2  MoneyDeposited   (+1000)       balance = 1000
v3  MoneyWithdrawn   (-300)        balance = 700
v4  MoneyDeposited   (+400)        balance = 1100
v5  MoneyWithdrawn   (-600)        balance = 500
```

Final Balance = 500 (derived, not stored)

KEY BENEFITS

- Full audit trail of every change
- Temporal queries and debugging
- Replay events to rebuild any state
- Stronger domain model and invariants
- Great for financial, compliance, and complex workflows

TRADE-OFFS

- More complex than CRUD
- Event schema evolution is hard
- Rebuild can be slow without snapshots
- Storage grows with time
- Operational complexity (monitoring, retention, projections)

INTERVIEW TAKEAWAYS

- Events are the source of truth
- Never update events, only append
- Rebuild must be deterministic and idempotent
- Snapshots improve performance, not correctness
- Optimistic concurrency protects consistency
- Pairs naturally with CQRS and projections

WHEN TO USE EVENT SOURCING

- Auditability is critical
- You need full history of changes
- Replay and time travel are valuable
- Complex domain with rich invariants
- Regulatory or compliance requirements

PERFECT PAIRINGS

- CQRS
- Event Outbox
- Saga Pattern
- Projection Pipeline

REMEMBER

Store events.
Trust events.
Rebuild anything.
That's Event Sourcing. 💪

🚀 BUILDING SYSTEMS. LEARNING EVERYDAY. SHARING IN PUBLIC. 💜

FOLLOW THE JOURNEY AND LEVEL UP YOUR SYSTEM DESIGN SKILLS!

Day 21: Event Sourcing, Snapshotting, and Aggregate Rebuild

Day 21 focuses on storing domain changes as immutable events so current state can be reconstructed, audited, and replayed reliably.

It covers:

- event sourcing fundamentals
- append-only event store design
- aggregate rebuild from event history
- snapshotting for faster rehydration
- optimistic concurrency with expected versions
- replay and recovery trade-offs

In short, Day 21 is about making events the source of truth and deriving state from them safely.

Core ideas

Why Event Sourcing exists

- Traditional CRUD keeps only latest state and loses change history details.
- Event sourcing stores each business change as a durable domain event.
- State is derived by replaying events in order.

Event store model

- Events are immutable and appended, never updated in place.
- Each aggregate stream has a version to preserve ordering.
- Writes commonly use optimistic concurrency (expected_version) to prevent lost updates.

Aggregate rebuild

- Load all events for an aggregate and apply them in sequence.
- Business invariants are enforced in command handling before appending events.
- Rebuild logic should be deterministic and idempotent.

Snapshotting

- Long event streams can make rebuild expensive.
- Snapshots persist aggregate state at a known version/sequence.
- Rehydrate by loading snapshot first, then replaying only tail events.

Trade-offs

- Event sourcing improves auditability, replay, and temporal debugging.

- It adds complexity in schema evolution and projection maintenance.
- Strong event contracts and versioning strategy are essential.

Day-21 sample: Event-Sourced Account Ledger

This repository includes a small event-sourcing sample in both Python and Java.

Functional requirements

- Open an account with zero balance
- Deposit and withdraw funds with validation
- Persist events in append-only order
- Rebuild account state from events
- Create and use snapshots for faster restore

High-level components

- EventStore: append-only event persistence with sequence numbers
- SnapshotStore: latest snapshot per account
- AccountAggregate: applies events to compute state
- CommandService: validates commands and appends events

Data flow

1. Client issues command (open_account, deposit, withdraw)
2. Service rehydrates aggregate from snapshot + tail events
3. Service validates business rules
4. Service appends new domain event with expected version
5. Aggregate state can be rebuilt anytime via replay

Project structure

```
Day-21/
  README.md
  event-sourcing-sample/
    python/
      event_sourcing_account.py
      demo.py
    java/
      EventSourcingAccount.java
      Main.java
```

Run the sample

Python

```
cd event-sourcing-sample\python
python demo.py
```

Java

```
cd event-sourcing-sample\java
javac Main.java EventSourcingAccount.java
java Main
```

Interview takeaways

- Event sourcing makes historical behavior and auditing first-class
- Rebuild logic and event ordering correctness are critical
- Snapshots improve performance but must align with event sequence/version
- Optimistic concurrency protects aggregate consistency under parallel writes
- CQRS read models can be fed directly from event streams

Next improvements

- Add event schema versioning and upcasters
- Add idempotency keys for command deduplication
- Add stream partitioning and retention strategy
- Add projection pipeline from events to query models
- Add monitoring for snapshot lag and replay latency

Code Appendix

Python and Java source used in Day 21 sample.

event-sourcing-sample\python\event_sourcing_account.py

```python
from __future__ import annotations

from dataclasses import dataclass


@dataclass(frozen=True)
class DomainEvent:
    sequence: int
    account_id: str
    event_type: str
    amount_cents: int
    version: int


@dataclass(frozen=True)
class AccountSnapshot:
    account_id: str
    balance_cents: int
    version: int
    last_sequence: int


class EventStore:
    def __init__(self) -> None:
        self._events: list[DomainEvent] = []
        self._next_sequence = 1

    def append(
        self,
        account_id: str,
        event_type: str,
        amount_cents: int,
        expected_version: int,
    ) -> DomainEvent:
        current_version = self.latest_version(account_id)
        if current_version != expected_version:
            raise ValueError(
                f"version mismatch for {account_id}: expected {expected_version}, got {current_version}"
            )

        event = DomainEvent(
            sequence=self._next_sequence,
            account_id=account_id,
            event_type=event_type,
            amount_cents=amount_cents,
            version=current_version + 1,
        )
        self._events.append(event)
        self._next_sequence += 1
        return event

    def latest_version(self, account_id: str) -> int:
        versions = [e.version for e in self._events if e.account_id == account_id]
        return max(versions) if versions else 0

    def events_for_account(self, account_id: str, after_sequence: int = 0) -> list[DomainEvent]:
        return [
            e
            for e in self._events
            if e.account_id == account_id and e.sequence > after_sequence
        ]

    def all_events(self) -> list[DomainEvent]:
        return self._events[:]
```

```python
class SnapshotStore:
    def __init__(self) -> None:
        self._snapshots: dict[str, AccountSnapshot] = {}

    def put(self, snapshot: AccountSnapshot) -> None:
        self._snapshots[snapshot.account_id] = snapshot

    def get(self, account_id: str) -> AccountSnapshot | None:
        return self._snapshots.get(account_id)


class AccountAggregate:
    def __init__(self, account_id: str) -> None:
        self.account_id = account_id
        self.is_open = False
        self.balance_cents = 0
        self.version = 0
        self.last_sequence = 0

    def apply(self, event: DomainEvent) -> None:
        if event.event_type == "AccountOpened":
            self.is_open = True
        elif event.event_type == "FundsDeposited":
            self.balance_cents += event.amount_cents
        elif event.event_type == "FundsWithdrawn":
            self.balance_cents -= event.amount_cents
        else:
            raise ValueError(f"unknown event_type: {event.event_type}")

        self.version = event.version
        self.last_sequence = event.sequence

    @staticmethod
    def rehydrate(
        account_id: str,
        events: list[DomainEvent],
        snapshot: AccountSnapshot | None = None,
    ) -> AccountAggregate:
        agg = AccountAggregate(account_id)
        if snapshot is not None:
            agg.is_open = True
            agg.balance_cents = snapshot.balance_cents
            agg.version = snapshot.version
            agg.last_sequence = snapshot.last_sequence

        for event in events:
            agg.apply(event)
        return agg


class CommandService:
    def __init__(self, event_store: EventStore, snapshot_store: SnapshotStore) -> None:
        self._event_store = event_store
        self._snapshot_store = snapshot_store

    def _load(self, account_id: str) -> AccountAggregate:
        snapshot = self._snapshot_store.get(account_id)
        after = snapshot.last_sequence if snapshot else 0
        tail_events = self._event_store.events_for_account(account_id, after_sequence=after)
        return AccountAggregate.rehydrate(account_id, tail_events, snapshot=snapshot)

    def open_account(self, account_id: str) -> DomainEvent:
        agg = self._load(account_id)
        if agg.is_open:
            raise ValueError(f"account already open: {account_id}")
        return self._event_store.append(
            account_id=account_id,
            event_type="AccountOpened",
            amount_cents=0,
            expected_version=agg.version,
        )
```

```
    def deposit(self, account_id: str, amount_cents: int) -> DomainEvent:
        if amount_cents <= 0:
            raise ValueError("amount_cents must be positive")
        agg = self._load(account_id)
        if not agg.is_open:
            raise ValueError(f"account not open: {account_id}")
        return self._event_store.append(
            account_id=account_id,
            event_type="FundsDeposited",
            amount_cents=amount_cents,
            expected_version=agg.version,
        )

    def withdraw(self, account_id: str, amount_cents: int) -> DomainEvent:
        if amount_cents <= 0:
            raise ValueError("amount_cents must be positive")
        agg = self._load(account_id)
        if not agg.is_open:
            raise ValueError(f"account not open: {account_id}")
        if agg.balance_cents < amount_cents:
            raise ValueError("insufficient balance")
        return self._event_store.append(
            account_id=account_id,
            event_type="FundsWithdrawn",
            amount_cents=amount_cents,
            expected_version=agg.version,
        )

    def snapshot(self, account_id: str) -> AccountSnapshot:
        agg = self._load(account_id)
        if not agg.is_open:
            raise ValueError(f"account not open: {account_id}")
        snapshot = AccountSnapshot(
            account_id=account_id,
            balance_cents=agg.balance_cents,
            version=agg.version,
            last_sequence=agg.last_sequence,
        )
        self._snapshot_store.put(snapshot)
        return snapshot

    def rebuild_full(self, account_id: str) -> AccountAggregate:
        events = self._event_store.events_for_account(account_id)
        return AccountAggregate.rehydrate(account_id, events, snapshot=None)

    def rebuild_with_snapshot(self, account_id: str) -> AccountAggregate:
        snapshot = self._snapshot_store.get(account_id)
        after = snapshot.last_sequence if snapshot else 0
        tail = self._event_store.events_for_account(account_id, after_sequence=after)
        return AccountAggregate.rehydrate(account_id, tail, snapshot=snapshot)
```

event-sourcing-sample\python\demo.py

```
from event_sourcing_account import CommandService, EventStore, SnapshotStore


def print_aggregate(prefix: str, agg) -> None:
    print(
        prefix,
        f"account={agg.account_id} balance={agg.balance_cents} version={agg.version} seq={agg.last_sequence
    )


def main() -> None:
    events = EventStore()
    snapshots = SnapshotStore()
    commands = CommandService(events, snapshots)

    account_id = "acct-100"

    commands.open_account(account_id)
```

```
    commands.deposit(account_id, 10_000)
    commands.deposit(account_id, 2_500)
    commands.withdraw(account_id, 1_200)

    full_before_snapshot = commands.rebuild_full(account_id)
    print_aggregate("before snapshot:", full_before_snapshot)

    snap = commands.snapshot(account_id)
    print(
        "snapshot created:",
        f"balance={snap.balance_cents} version={snap.version} seq={snap.last_sequence}",
    )

    commands.deposit(account_id, 500)
    commands.withdraw(account_id, 300)

    full = commands.rebuild_full(account_id)
    with_snapshot = commands.rebuild_with_snapshot(account_id)

    print_aggregate("full rebuild:", full)
    print_aggregate("snapshot rebuild:", with_snapshot)

    print("event count:", len(events.all_events()))
    print("state match:", full.balance_cents == with_snapshot.balance_cents)


if __name__ == "__main__":
    main()
```

event-sourcing-sample\java\EventSourcingAccount.java

```
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

public class EventSourcingAccount {
    public static final class DomainEvent {
        private final int sequence;
        private final String accountId;
        private final String eventType;
        private final int amountCents;
        private final int version;

        public DomainEvent(int sequence, String accountId, String eventType, int amountCents, int version)
            this.sequence = sequence;
            this.accountId = accountId;
            this.eventType = eventType;
            this.amountCents = amountCents;
            this.version = version;
        }

        public int getSequence() {
            return sequence;
        }

        public String getAccountId() {
            return accountId;
        }

        public String getEventType() {
            return eventType;
        }

        public int getAmountCents() {
            return amountCents;
        }

        public int getVersion() {
            return version;
        }
```

```
}

public static final class AccountSnapshot {
    private final String accountId;
    private final int balanceCents;
    private final int version;
    private final int lastSequence;

    public AccountSnapshot(String accountId, int balanceCents, int version, int lastSequence) {
        this.accountId = accountId;
        this.balanceCents = balanceCents;
        this.version = version;
        this.lastSequence = lastSequence;
    }

    public String getAccountId() {
        return accountId;
    }

    public int getBalanceCents() {
        return balanceCents;
    }

    public int getVersion() {
        return version;
    }

    public int getLastSequence() {
        return lastSequence;
    }
}

public static final class EventStore {
    private final List<DomainEvent> events = new ArrayList<>();
    private int nextSequence = 1;

    public DomainEvent append(
        String accountId,
        String eventType,
        int amountCents,
        int expectedVersion
    ) {
        int currentVersion = latestVersion(accountId);
        if (currentVersion != expectedVersion) {
            throw new IllegalArgumentException(
                "version mismatch for "
                    + accountId
                    + ": expected "
                    + expectedVersion
                    + ", got "
                    + currentVersion
            );
        }

        DomainEvent event = new DomainEvent(
            nextSequence,
            accountId,
            eventType,
            amountCents,
            currentVersion + 1
        );
        events.add(event);
        nextSequence += 1;
        return event;
    }

    public int latestVersion(String accountId) {
        int version = 0;
        for (DomainEvent event : events) {
            if (accountId.equals(event.getAccountId()) && event.getVersion() > version) {
                version = event.getVersion();
```

```
            }
        }
        return version;
    }

    public List<DomainEvent> eventsForAccount(String accountId, int afterSequence) {
        List<DomainEvent> result = new ArrayList<>();
        for (DomainEvent event : events) {
            if (!accountId.equals(event.getAccountId())) {
                continue;
            }
            if (event.getSequence() <= afterSequence) {
                continue;
            }
            result.add(event);
        }
        return result;
    }

    public List<DomainEvent> allEvents() {
        return new ArrayList<>(events);
    }
}

public static final class SnapshotStore {
    private final Map<String, AccountSnapshot> snapshots = new LinkedHashMap<>();

    public void put(AccountSnapshot snapshot) {
        snapshots.put(snapshot.getAccountId(), snapshot);
    }

    public AccountSnapshot get(String accountId) {
        return snapshots.get(accountId);
    }
}

public static final class AccountAggregate {
    private final String accountId;
    private boolean open;
    private int balanceCents;
    private int version;
    private int lastSequence;

    public AccountAggregate(String accountId) {
        this.accountId = accountId;
        this.open = false;
        this.balanceCents = 0;
        this.version = 0;
        this.lastSequence = 0;
    }

    public void apply(DomainEvent event) {
        switch (event.getEventType()) {
            case "AccountOpened":
                open = true;
                break;
            case "FundsDeposited":
                balanceCents += event.getAmountCents();
                break;
            case "FundsWithdrawn":
                balanceCents -= event.getAmountCents();
                break;
            default:
                throw new IllegalArgumentException("unknown eventType: " + event.getEventType());
        }
        version = event.getVersion();
        lastSequence = event.getSequence();
    }

    public static AccountAggregate rehydrate(
        String accountId,
```

```
        List<DomainEvent> events,
        AccountSnapshot snapshot
    ) {
        AccountAggregate aggregate = new AccountAggregate(accountId);
        if (snapshot != null) {
            aggregate.open = true;
            aggregate.balanceCents = snapshot.getBalanceCents();
            aggregate.version = snapshot.getVersion();
            aggregate.lastSequence = snapshot.getLastSequence();
        }

        for (DomainEvent event : events) {
            aggregate.apply(event);
        }
        return aggregate;
    }

    public String getAccountId() {
        return accountId;
    }

    public boolean isOpen() {
        return open;
    }

    public int getBalanceCents() {
        return balanceCents;
    }

    public int getVersion() {
        return version;
    }

    public int getLastSequence() {
        return lastSequence;
    }
}

public static final class CommandService {
    private final EventStore eventStore;
    private final SnapshotStore snapshotStore;

    public CommandService(EventStore eventStore, SnapshotStore snapshotStore) {
        this.eventStore = eventStore;
        this.snapshotStore = snapshotStore;
    }

    private AccountAggregate load(String accountId) {
        AccountSnapshot snapshot = snapshotStore.get(accountId);
        int after = snapshot != null ? snapshot.getLastSequence() : 0;
        List<DomainEvent> tail = eventStore.eventsForAccount(accountId, after);
        return AccountAggregate.rehydrate(accountId, tail, snapshot);
    }

    public DomainEvent openAccount(String accountId) {
        AccountAggregate aggregate = load(accountId);
        if (aggregate.isOpen()) {
            throw new IllegalArgumentException("account already open: " + accountId);
        }
        return eventStore.append(accountId, "AccountOpened", 0, aggregate.getVersion());
    }

    public DomainEvent deposit(String accountId, int amountCents) {
        if (amountCents <= 0) {
            throw new IllegalArgumentException("amountCents must be positive");
        }
        AccountAggregate aggregate = load(accountId);
        if (!aggregate.isOpen()) {
            throw new IllegalArgumentException("account not open: " + accountId);
        }
        return eventStore.append(
```

```
                accountId,
                "FundsDeposited",
                amountCents,
                aggregate.getVersion()
            );
        }

        public DomainEvent withdraw(String accountId, int amountCents) {
            if (amountCents <= 0) {
                throw new IllegalArgumentException("amountCents must be positive");
            }
            AccountAggregate aggregate = load(accountId);
            if (!aggregate.isOpen()) {
                throw new IllegalArgumentException("account not open: " + accountId);
            }
            if (aggregate.getBalanceCents() < amountCents) {
                throw new IllegalArgumentException("insufficient balance");
            }
            return eventStore.append(
                accountId,
                "FundsWithdrawn",
                amountCents,
                aggregate.getVersion()
            );
        }

        public AccountSnapshot snapshot(String accountId) {
            AccountAggregate aggregate = load(accountId);
            if (!aggregate.isOpen()) {
                throw new IllegalArgumentException("account not open: " + accountId);
            }
            AccountSnapshot snapshot = new AccountSnapshot(
                accountId,
                aggregate.getBalanceCents(),
                aggregate.getVersion(),
                aggregate.getLastSequence()
            );
            snapshotStore.put(snapshot);
            return snapshot;
        }

        public AccountAggregate rebuildFull(String accountId) {
            return AccountAggregate.rehydrate(accountId, eventStore.eventsForAccount(accountId, 0), null);
        }

        public AccountAggregate rebuildWithSnapshot(String accountId) {
            AccountSnapshot snapshot = snapshotStore.get(accountId);
            int after = snapshot != null ? snapshot.getLastSequence() : 0;
            return AccountAggregate.rehydrate(accountId, eventStore.eventsForAccount(accountId, after), sna
        }
    }
}
```

event-sourcing-sample\java\Main.java

```
public class Main {
    private static void printAggregate(String prefix, EventSourcingAccount.AccountAggregate aggregate) {
        System.out.println(
            prefix
                + " account="
                + aggregate.getAccountId()
                + " balance="
                + aggregate.getBalanceCents()
                + " version="
                + aggregate.getVersion()
                + " seq="
                + aggregate.getLastSequence()
        );
    }

    public static void main(String[] args) {
```

```
        EventSourcingAccount.EventStore events = new EventSourcingAccount.EventStore();
        EventSourcingAccount.SnapshotStore snapshots = new EventSourcingAccount.SnapshotStore();
        EventSourcingAccount.CommandService commands = new EventSourcingAccount.CommandService(events, snap

        String accountId = "acct-100";

        commands.openAccount(accountId);
        commands.deposit(accountId, 10_000);
        commands.deposit(accountId, 2_500);
        commands.withdraw(accountId, 1_200);

        EventSourcingAccount.AccountAggregate beforeSnapshot = commands.rebuildFull(accountId);
        printAggregate("before snapshot:", beforeSnapshot);

        EventSourcingAccount.AccountSnapshot snapshot = commands.snapshot(accountId);
        System.out.println(
            "snapshot created: balance="
                + snapshot.getBalanceCents()
                + " version="
                + snapshot.getVersion()
                + " seq="
                + snapshot.getLastSequence()
        );

        commands.deposit(accountId, 500);
        commands.withdraw(accountId, 300);

        EventSourcingAccount.AccountAggregate full = commands.rebuildFull(accountId);
        EventSourcingAccount.AccountAggregate withSnapshot = commands.rebuildWithSnapshot(accountId);

        printAggregate("full rebuild:", full);
        printAggregate("snapshot rebuild:", withSnapshot);

        System.out.println("event count: " + events.allEvents().size());
        System.out.println("state match: " + (full.getBalanceCents() == withSnapshot.getBalanceCents()));
    }
}
```